LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL 5

STUKDAT



Disusun oleh:

[MUHAMMAD NAUFAL AKMAL PRABOWO]

109082500186

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL

Link Repo Github Modul X : (https://github.com/naufalakmall/struktur-data.git)

1. **Main.cpp**

   Source Code

```
#include <iostream>
#include "Singlylist.h"

using namespace std;

int main()
{
    List L;
    address P1, P2, P3, P4, P5;
    P1 = P2 = P3 = P4 = P5 = Nil;

    // 1. Inisialisasi List
    createList(L);

    // 2. Alokasi dan Insert Data (Urutan dimasukkan agar
membentuk: 9 -> 12 -> 8 -> 0 -> 2)
    P1 = alokasi(2);
    insertFirst(L, P1);

    P2 = alokasi(0);
    insertFirst(L, P2);

    P3 = alokasi(8);
    insertFirst(L, P3);

    P4 = alokasi(12);
    insertFirst(L, P4);

    P5 = alokasi(9);
    insertFirst(L, P5);

    // Output Soal No. 2: Cetak seluruh isi list
    printInfo(L);

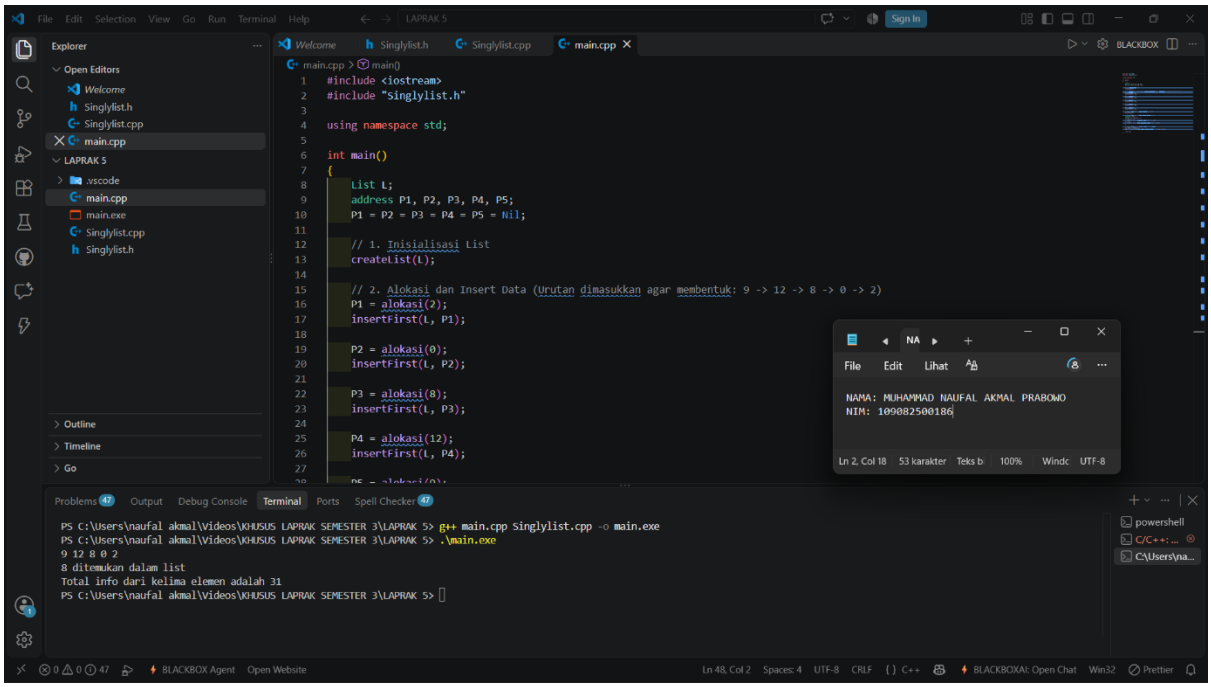
    // Output Soal No. 3: Cari elemen dengan info 8
    infotype x = 8;
    address P = findElm(L, x);
    if (P != Nil) {
        cout << x << " ditemukan dalam list" << endl;
    } else {
```

```
            cout << x << " tidak ditemukan dalam list" << endl;
        }

        // Output Soal No. 4: Hitung jumlah total elemen
        int total = sumInfo(L);
        cout << "Total info dari kelima elemen adalah " << total
    << endl;

        return 0;
    }
```

Screenshoot program :



Deskripsi program

File Singlylist.h berfungsi sebagai file header yang mendefinisikan struktur data Abstract Data Type (ADT) Singly Linked List, meliputi pemetaan tipe data infotype dan pointer address, struktur simpul ElmList (memuat variabel info dan pointer next), serta struktur List yang menampung pointer First. Selain itu, file ini menampung seluruh deklarasi prototipe fungsi dan prosedur dasar manajemen linked list, seperti pembentukan list, alokasi/dealokasi memori, penyisipan elemen di awal (insertFirst), penayangan data (printInfo), pencarian elemen (findElm), hingga penjumlahan total elemen (sumInfo).

2. **Singlylist.cpp**

Source Code

```
#include "Singlylist.h"

void createList(List &L) {
    L.First = Nil;
}

address alokasi(infotype x) {
    address P = new ElmList;
    if (P != Nil) {
        P->info = x;
        P->next = Nil;
    }
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = Nil;
}

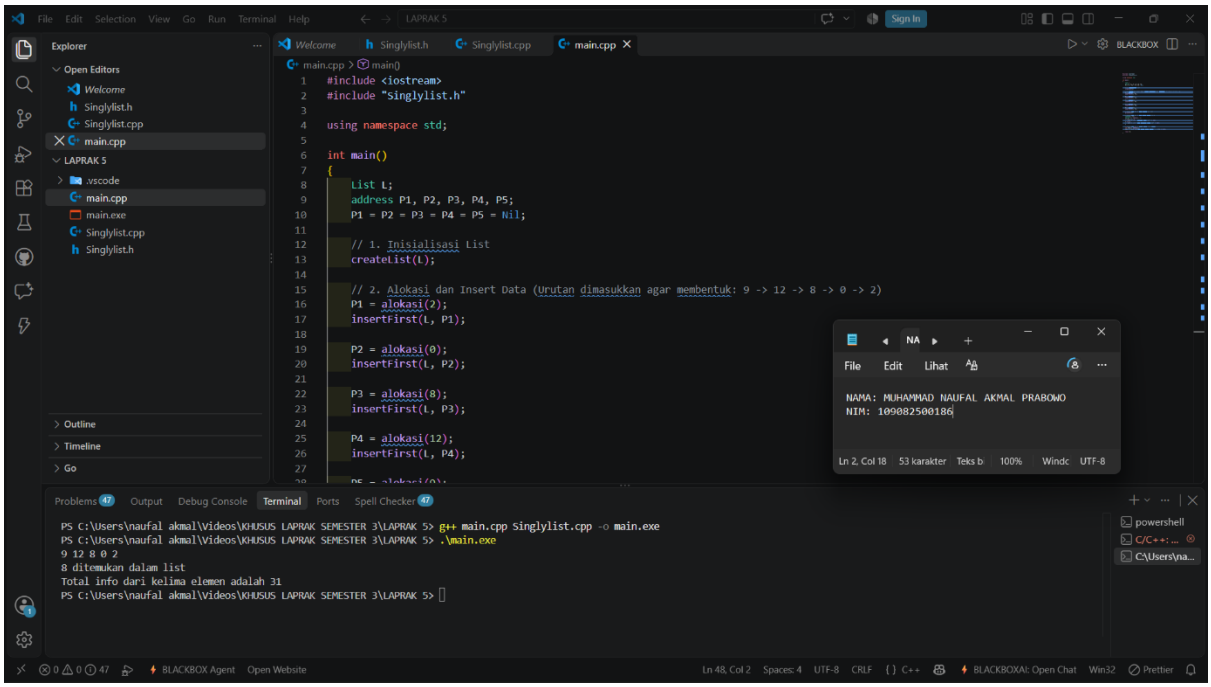
void printInfo(List L) {
    address P = L.First;
    while (P != Nil) {
        cout << P->info << " ";
        P = P->next;
    }
    cout << endl;
}

void insertFirst(List &L, address P) {
    if (P != Nil) {
        P->next = L.First;
        L.First = P;
    }
}

address findElm(List L, infotype x) {
    address P = L.First;
    while (P != Nil) {
        if (P->info == x) {
            return P;
        }
        P = P->next;
    }
    return Nil;
}
```

```
int sumInfo(List L) {
    int total = 0;
    address P = L.First;
    while (P != Nil) {
        total += P->info;
        P = P->next;
    }
    return total;
}
```

Screenshoot program



Deskripsi program

File Singlylist.cpp memuat implementasi kode utama dari seluruh pustaka fungsi dan prosedur yang telah dideklarasikan sebelumnya di file Singlylist.h. Di dalam file ini dijalankan logika alokasi memori menggunakan operator new dan dealokasi dengan delete, penautan simpul baru di posisi terdepan pada insertFirst, penelusuran (traversal) rantai simpul untuk mencetak seluruh elemen (printInfo) maupun mencari simpul dengan nilai tertentu (findElm), serta perhitungan akumulasi nilai numerik dari seluruh simpul terhubung melalui fungsi sumInfo.

3. **Singlylist.h**

Source Code

```
#ifndef SINGLYLIST_H
#define SINGLYLIST_H

#include <iostream>
using namespace std;

#define Nil NULL

typedef int infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
};

struct List {
    address First;
};

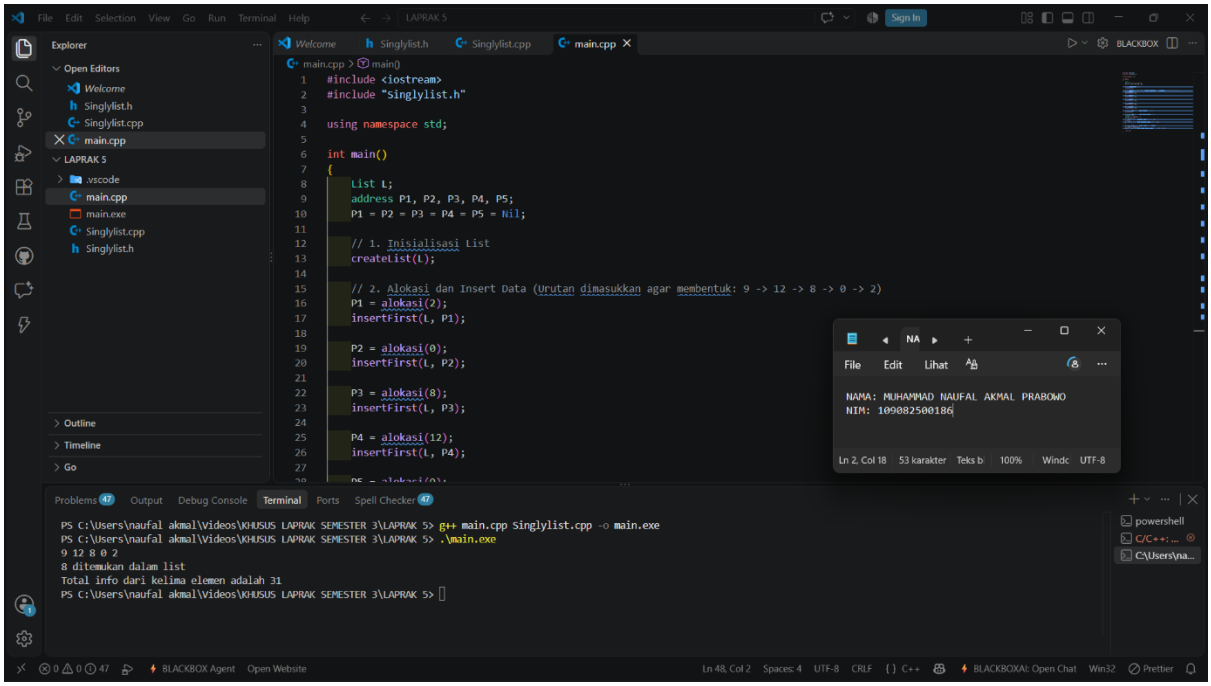
// Prosedur & Fungsi Dasar Linked List (Soal No. 2)
void createList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void printInfo(List L);
void insertFirst(List &L, address P);

// Fungsi Pencarian Elemen (Soal No. 3)
address findElm(List L, infotype x);

// Fungsi Menghitung Total Info (Soal No. 4)
int sumInfo(List L);

#endif
```

Screenshoot program



Deskripsi program

File main.cpp bertindak sebagai titik masuk utama (main entry point) program yang menguji seluruh fungsionalitas ADT Singly Linked List yang telah dibuat. Fungsi main() menginisialisasi variabel list L, melakukan alokasi dan penyisipan data angka (2, 0, 8, 12, dan 9) secara berurutan menggunakan insertFirst sehingga terbentuk struktur 9 -> 12 -> 8 -> 0 -> 2, kemudian menampilkan susunan angka tersebut ke layar terminal, menjalankan proses pencarian nilai 8, dan mencetak total penjumlahan seluruh isi elemen.

GUIDED

1. SingleLinkedList.cpp

Source Code

```
#include  "SingleLinkedList.h"
#include <iostream>
using namespace std;

// Membuat list kosong
void CreateList(List &L) {
    L.First = nullptr;
}

// Membuat node baru
ElmList* alokasi(int nilai) {
    ElmList* P = new ElmList;
```

```
    P->info = nilai;
    P->next = nullptr;

    return P;
}

// Membebaskan memori node
void dealokasi(ElmList* &P) {
    delete P;
    P = nullptr;
}

// Menampilkan semua isi list
void printInfo(List L) {
    ElmList* P = L.First;

    while (P != nullptr) {
        cout << P->info << " ";
        P = P->next;
    }

    cout << endl;
}

// Mencari node berdasarkan nilai
ElmList* findElm(List L, int nilai) {
    ElmList* P = L.First;

    while (P != nullptr) {
        if (P->info == nilai) {
            return P;
        }
        P = P->next;
    }

    return nullptr;
}

// Menambahkan node di awal
void insertFirst(List &L, int nilai) {
    ElmList* P = alokasi(nilai);

    P->next = L.First;
    L.First = P;
}

// Menambahkan node di akhir
void insertLast(List &L, int nilai) {
```

```
    ElmList* P = alokasi(nilai);
    // Mencari node berdasarkan nilai
}
    ElmList* findElm(List L, int nilai) {
    ElmList* P = L.First;

    while (P != nullptr) {
        if (P->info == nilai) {
            return P;
        }

        P = P->next;
    }

    return nullptr;
}

// Menambahkan node di awal
void insertFirst(List &L, int nilai) {
    ElmList* P = alokasi(nilai);

    P->next = L.First;
    L.First = P;
}

// Menambahkan node di akhir
void insertLast(List &L, int nilai) {
    ElmList* P = alokasi(nilai);

    if (L.First == nullptr) {
        L.First = P;
    } else {
        ElmList* Q = L.First;

        while (Q->next != nullptr) {
            Q = Q->next;
        }

        Q->next = P;
    }
}

// Menambahkan node setelah nilai tertentu
void insertAfter(List &L, int nilaiSebelum, int nilaiBaru) {
    ElmList* Q = findElm(L, nilaiSebelum);

    if (Q == nullptr) {
        cout << "Nilai sebelumnya tidak ditemukan." << endl;
```

```
        } else {
            ElmList* P = alokasi(nilaiBaru);
        }
```

2. SingleLinkedList.h

Source Code

```
#ifndef SLL_H
#define SLL_H

struct ElmList {
    int info;
    ElmList* next;
};

struct List {
    ElmList* First;
};

void CreateList(List &L);
ElmList* alokasi(int nilai);

void dealokasi(ElmList* &P);
void printInfo(List L);

ElmList* findElm(List L, int nilai);

ElmList* findElm(List L, int nilai);

void insertFirst(List &L, int nilai);
void insertLast(List &L, int nilai);
void insertAfter(List &L, int nilaiSebelum, int nilaiBaru);

void delFirst(List &L);
void delLast(List &L);
void delAfter(List &L, int nilaiSebelum);
void delP(List &L, int nilai);

void updateList(List &L, int nilaiLama, int nilaiBaru);
int nbList(List L);
void delAll(List &L);

#endif
```